

K8S introduction

Containers vs Virtual machines

Virtual Machines:

- virtualized compute (emulation)
- first gen cloud
- hypervisor: separate VMs + resource allocation
- separated OS
- heavy (Gbs)

Containers:

- light (Mbs)
- application layer abstraction
- shared OS kernel
- isolated user spaces
- scaling and faster boot
- more efficient
- all dependencies included

Problem when you have a lot of them...

K8S

What is K8s?

- container orchestrator

It **automates**:

- Running containers
- Restarting failed apps
- Scaling up/down
- Networking & discovery
- Deploying updates safely

Architecture

Control plane

- kube-api server
- etcd
- kube-scheduler
- kube-controller-manager
- (cloud-controller-manager)

Building blocks

- cluster
- nodes
- pods
- containers

Describing objects

- every object has a spec (yaml)
- desired state
- contains all information
- declarative vs imperative

Resources

Namespace

- virtual environment
- resource isolation

Pods

- smallest unit
- groups containers
- shared resources and context (files)

Workloads

Replicaset

- Watches the cluster
- If a Pod dies → creates a new one
- If you scale up → adds Pods
- If you scale down → removes Pods
- rarely used directly

Deployment

Higher-level abstraction that **manages ReplicaSets** and gives you:

- Rolling updates
- Rollbacks
- History tracking

Daemonset

- Automatically deploys a Pod to **every node**
- If a new node joins, a Pod is scheduled there too
- If a node leaves, its Pod is cleaned up

Used for Node-level background services, like:

- Logging agents (e.g. Fluentd, Filebeat)
- Monitoring (e.g. Prometheus node-exporter)
- Network tools (e.g. CNI plugins)

Jobs

- Creates one or more pods
- Will retry execution until completion
- Used to reliably run one Pod to completion
- Run a job on schedule => CronJob

Pod lifecycle

States

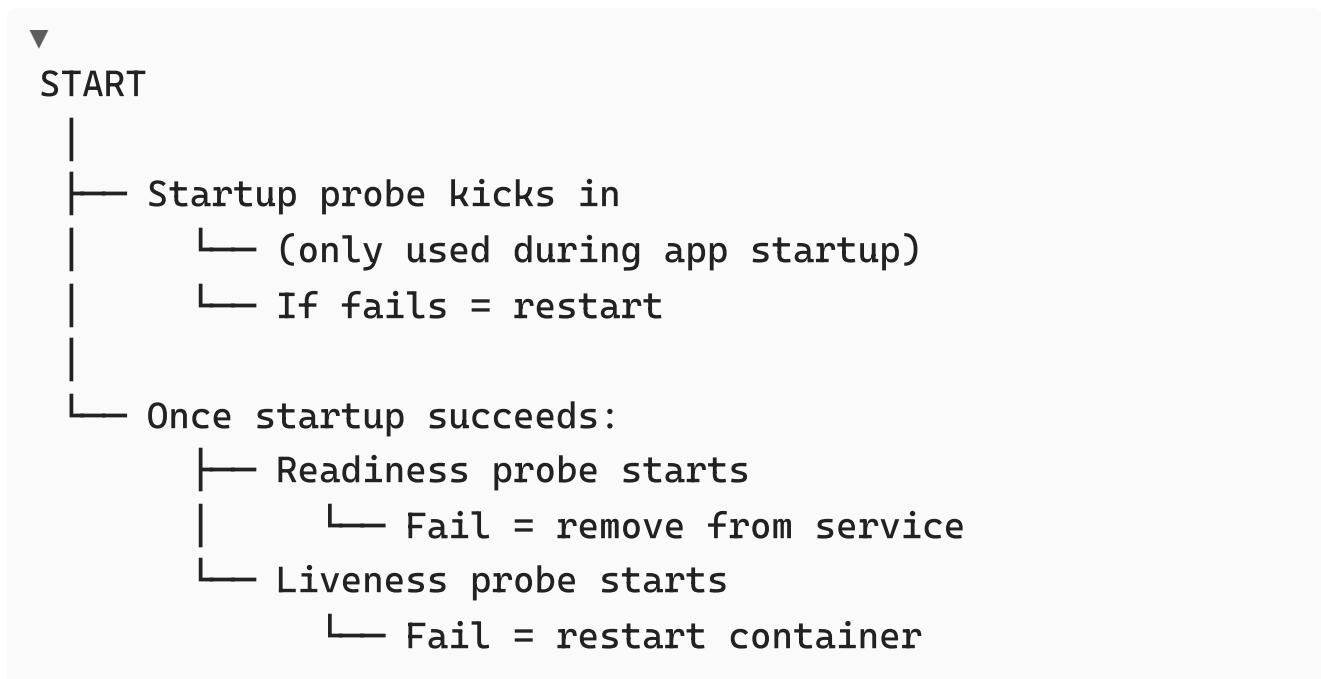
- pending
 - accepted by the scheduler
 - busy creating containers
- running
 - at least one container is running
 - pod bound to a node
- succeeded
 - all containers successfully completed (0 status)
- failed
 - one or more containers exited with a non 0 status

Lifecycle hooks

- container lifecycle
 - waiting
 - running
 - terminated

- available hooks
 - preStop
 - graceful shutdown logic
 - k8s waits for hook to finish for `terminationGracePeriodSeconds`
 - postStart
 - after container is created
 - load configuration
 - signal other services
 - preload data

Probes



- probe methods
 - httpGet: http request to endpoint
 - tcpSocket: open tcp connection to port
 - exec: run a command inside the container
- startup probe
 - is the application ready?
- readiness probe

- is the pod ready to accept traffic
- fail: container is removed from Service load balancer
- liveness probe
 - is the container still alive
 - fail: container killed and restarted

Services

- why services
 - pods can be created and destroyed at any time
 - IP addresses change
- abstraction on top of pods
- provide a stable virtual endpoint (IP + DNS) to reach a group of selected pods
- building blocks
 - selector: match pods based on labels to route traffic to
 - port mapping: what port does the service need to listen on and on what port are the pods listening
 - type: how is the service exposed?
 - ClusterIP
 - default
 - cluster-internal IP
 - internal communication
 - NodePort
 - outside access
 - exposes on predefined port on all nodes
 - creates a ClusterIP in the background
 - Loadbalancer
 - use cloud provider Load Balancer
 - creates NodePort and ClusterIP service
 - ExternalName

- access to external service from within the cluster
- DNS